iLipi Learning · Take-home assignment — MiniLearn

1 message

<tech@ilipilearning.com> Fri, 5 Jun, 2026 at 2:53 pm
To: Akhilesh Mani Tiwari <akhileshmanitiwari419@gmail.com>
Cc: hp@ilipilearning.com, subhodip@ilipilearning.com

Hi Akhilesh,

Below is a small technical exercise to help us finalise our decision. Read everything before starting. If something is unclear, reply to this email — we answer within a working day.

TIME BUDGET

Aim for 10-15 working hours TOTAL over 5 days. Stop when you hit the budget even if something is incomplete — list what's TODO in your README. We are NOT looking for perfection; we are looking for clean code, clear decisions, and an honest README.

Deadline: end of day Tuesday 10 June 2026, IST.

THE BRIEF — "MINILEARN"

Build the backend + a minimal frontend for MiniLearn, a stripped-down Udemy-style course platform. An instructor or admin uses your API to manage courses and lessons. A learner uses it to browse, enroll, mark lessons complete, and attempt quizzes. There is one AI feature: an endpoint that auto-generates quiz questions from a lesson's content using an LLM.

That's the scope. Smaller than real Udemy on purpose.

STACK REQUIRED

Backend
- Node.js 20 + Express 4 (ESM modules)
- PostgreSQL (preferred) OR SQLite (acceptable if Postgres is heavy to set up locally — note the choice in README)
- Knex for migrations + queries, OR raw SQL files if you prefer
- TypeScript is a nice-to-have on the backend; plain JS ESM is fine
- express-validator (or zod) for input validation

LLM provider
- Your choice: Anthropic Claude, OpenAI, or Hugging Face Inference (free tier). If you have credits anywhere, use them. Hugging Face's free inference tier is enough for this scope.

Frontend (minimal)
- A small React + TypeScript UI that consumes your API
- Polished UI is NOT required — we'll judge the backend more heavily. A clean basic UI is fine.

WHAT TO BUILD — DATABASE

One migration (Knex or plain SQL) that creates these tables:

```
  users
    id              uuid PK
    email           text UNIQUE
    name            text
    role            enum('learner','instructor','admin')
    created_at      timestamp

  courses
    id              uuid PK
    title           text NOT NULL
    category        text
    headline        text
    instructor_id   FK -> users.id
    rating          numeric
    created_at      timestamp

  sections
    id              uuid PK
    course_id       FK -> courses.id  (ON DELETE CASCADE)
    title           text
    order           int

  lessons
    id              uuid PK
    section_id      FK -> sections.id (ON DELETE CASCADE)
    title           text
    content         text    -- plain text body
    duration_minutes  int
    order           int

  enrollments
    id              uuid PK
    user_id         FK -> users.id
    course_id       FK -> courses.id
    enrolled_at     timestamp
    UNIQUE (user_id, course_id)

  lesson_progress
    id              uuid PK
    user_id         FK
    lesson_id       FK
    completed       bool
    completed_at    timestamp
    UNIQUE (user_id, lesson_id)

  quiz_questions
    id              uuid PK
    course_id       FK -> courses.id
    question        text
    options         jsonb       -- array of 4 strings
    correct_answer  text        -- must exact-match one option
    source          enum('manual','ai')
    created_at      timestamp
```

Seed 2-3 courses with a few sections + lessons + quiz questions
so the API has data to return before any AI generation runs.

For ALL endpoints below, use a hard-coded user id "demo-user" for
the "current user". We are NOT asking you to implement real auth
— just add a one-paragraph note in the README on how you'd wire
it (JWT in httpOnly cookie OR Bearer header, your call).

WHAT TO BUILD — REST API

All return JSON. Use express-validator (or zod) for input

validation. Return proper HTTP status codes (400 for validation, 404 for missing, 502/503 for upstream AI failure, 500 only for genuine server bugs).

GET /api/courses
List courses with id, title, category, instructor name, rating, sections-count, lessons-count.
Query params: ?category=<string> ?search=<string>

GET /api/courses/:id
Single course with nested sections + lessons (no quiz).

POST /api/courses
Create a course. Body: { title, category, headline, instructor_id }. Returns 201 with the new row.

POST /api/courses/:id/sections
Append a section. Body: { title }. Auto-assigns order.

POST /api/sections/:id/lessons
Append a lesson. Body: { title, content, durationMinutes }.

POST /api/courses/:id/enroll
Idempotent. Returns 200 if already enrolled, 201 if new.

POST /api/lessons/:id/complete
Marks the demo-user's lesson_progress row complete.

GET /api/courses/:id/progress
Returns { totalLessons, completedLessons, percent } for the demo-user.

GET /api/courses/:id/quiz
Returns the persisted quiz questions for the course (manual + AI-generated combined). Hide correct_answer from the response (avoid client-side cheating).

POST /api/courses/:id/quiz/attempt
Body: { answers: [{ questionId, selectedOption }, ...] }
Grades server-side. Returns { score, total, passed } where passed = score/total >= 0.7. Persist the attempt in a small "quiz_attempts" table you add to the same migration.

POST /api/courses/:id/quiz/generate-from-lessons (AI)
Body: { count: number (3-10), lessonIds?: string[] }
Pulls the chosen lessons' content (or all lessons if lessonIds is omitted), prompts the LLM, parses the response, persists the generated questions in quiz_questions with source='ai', returns the new questions.

LLM PROMPT DESIGN — THE INTERESTING BIT

Your system prompt must:

- Constrain output to STRICT JSON with this shape:
  { "questions": [
    { "question": "...",
      "options": ["...","...","...","..."],
      "correctAnswer": "..." }] }
- Require EXACTLY 4 options per question.
- Require correctAnswer to match one option string EXACTLY.
- Ask for varied difficulty (mix of recall + application).
- Ask for plausible distractors — not throwaway nonsense.

Validate the JSON the model returns:
- Parse, check shape, check each question has 4 options and a

matching correctAnswer.
- Drop malformed questions silently (don't fail the batch).
- If validation leaves you with fewer than 3 valid questions, retry the LLM call ONCE with an explicit "you returned invalid JSON / wrong shape — try again" follow-up.
- If STILL bad after the retry, return 502 with a clear message. Never silently store garbage.

Other LLM hygiene:
- 30-second timeout on the LLM call.
- Log every call to console: provider, model, latency, approximate input/output token count.
- Truncate lesson content fed to the prompt to a sensible cap (e.g., 4000 chars per lesson) and note the truncation in the log line.

PRODUCTION-QUALITY DETAILS

- Environment variables in .env, never committed. Provide a .env.example.
- Rate-limit the AI generation endpoint with express-rate-limit (e.g., 10 calls per IP per hour). NOT all routes — just the expensive AI one.
- Return 503 with a clear "AI provider not configured" message if the LLM API key is missing — not 500.
- One Jest (or vitest) integration test that hits GET /api/courses against a test database (sqlite in-memory is fine for tests even if you use Postgres for dev).
- One small unit test for your JSON-validation helper.

MINIMAL FRONTEND

A simple React + TS UI that:

1. Lists courses from GET /api/courses with the search + category filter.
2. Course detail page that shows sections + lessons.
3. Click a lesson to see its content; "Mark complete" button.
4. Course detail page has a "Generate quiz with AI" form (count input + a Generate button) that calls the AI endpoint and shows a loading state.
5. After generation, a "Take quiz" button starts the quiz. One question at a time, "Submit" reveals correctness, end screen shows score + passed/failed.

Polished UI is NOT required here. We judge the backend more heavily than the UI. Clean basic Tailwind is fine.

README MUST INCLUDE

- How to run: Postgres setup OR sqlite fallback, step by step.
- LLM provider you used and why. How to get a key (link).
- Sample curl for every endpoint with example request + response.
- Costs / Limits section: estimated cost per generation for your provider, what happens if rate-limited, what you'd cache and where, what you'd index in the DB for scale.
- "If I had real auth" section: one paragraph on how you'd wire JWT cookies, refresh tokens, and role-based access for instructor-vs-learner endpoints.
- "What I'd do with another week" section.

DELIVERABLES

Reply to this email with:

1. The URL of your PUBLIC GitHub repo named "minilearn-akhilesh".
2. A 300-500 word summary in the email body covering:
   - What works and what doesn't.
   - Where you spent most of your time and what surprised you.
   - One thing you would have done differently with another week.
   - One question for the reviewer.
3. A 2-3 minute screen-recorded demo (Loom, YouTube unlisted, or Google Drive link). Show: a curl against your API, the AI-generation endpoint working live, and the small UI working end-to-end.

HOW WE EVALUATE

We rate three areas, 1-5:

1. Code quality
   File structure, error handling, type usage, separation of concerns. Schema design: keys, FK cascades, indexes. Not "is it clever" but "is it readable in 6 months".

2. Judgement
   LLM hygiene: prompt design, validation, retry, timeout, cost-awareness. Defensive coding: rate limit, secret handling, explicit 502/503 vs silent 500. Did you trade off the right things for the time budget?

3. Communication
   Commit messages, README, demo voice-over, email summary. We work async-first; how you write things is how you'll be 80% of the time.

A 3 in all three = "we'd extend an offer". A 4 anywhere = strong signal. A 2 anywhere is a red flag, but recoverable with discussion.

GROUND RULES

- You may use AI tools (Copilot, Cursor, Claude in chat) to help with code. You may NOT use AI to write your README or your demo voice-over. You must be able to explain every line of code in a follow-up call. We can tell when code was generated without understanding.
- Don't commit secrets. Use .env.example and document the variables.
- English for UI copy and comments.
- Commits should be small + frequent with conventional-style messages (feat:, fix:, chore:, refactor:). We look at the commit history, not only the final state.
- Keep this assignment confidential between us until a decision is made.

If you need an extra day for a legitimate reason , email us BEFORE the deadline. Not after.

Looking forward to your submission.

Best regards,
[Your name]
iLipi Learning
tech@ilipilearning.com